

CS675: Computer Vision Assignment 3

Due date: Apr 21st at 11:59pm.

Topic: Fine-tuning and Segmentation

Assignment Title: Object Segmentation

In this assignment, we'll learn how to segment objects from images using a pre-trained segmentation model, which saves us from building one from scratch. We'll work with a small dataset of balloon images, focusing specifically on segmenting the balloons as our target objects.

Instructions:

You will fine-tune a pre-trained **Mask R-CNN** model to adapt it to a new task with a custom set of object classes. A starter code in `PyTorch` is provided to help you get going, and using `PyTorch` is recommended for that reason. However, if you're more comfortable using `TensorFlow` or another deep learning framework, you're welcome to do so (as long as the core concepts of segmentation and fine-tuning are addressed).

Before you begin, take some time to understand the architecture and working principles of Mask R-CNN. Start by reading these two tutorials: [\[First Tutorial\]](#) and [\[Second Tutorial\]](#). For a deeper dive, you're also encouraged to explore the original [Mask R-CNN paper](#).

After understanding how Mask R-CNN works, let's start coding. `PyTorch`, through the `torchvision` library, provides a convenient model zoo with many popular pre-trained architectures ready to use. This makes it easy to get started with advanced models without building everything from scratch. For this assignment, we'll use a pre-trained Mask R-CNN model available in `torchvision`. Here's how you can load it:

```
import torch
import torchvision
from torchvision.models.detection import maskrcnn_resnet50_fpn
from torchinfo import summary

# Load pre-trained Mask R-CNN model
model = maskrcnn_resnet50_fpn(pretrained=True).eval()

summary(model, input_size=(1,3,224,224))
```

In the model summary, you'll notice the key components of the Mask R-CNN architecture: the **backbone with Feature Pyramid Network (FPN)** for feature extraction, the **Region Proposal Network (RPN)** for generating candidate object regions, and the **ROI heads**, which consist of two parts: one inherited from **Fast R-CNN** (`FastRCNNPredictor`) for classification and bounding box regression, and one specific to our model (`MaskRCNNPredictor`) for segmentation.

The pre-trained model was originally trained on the [COCO dataset](#), which includes 91 object classes. As a result, both the `FastRCNNPredictor` and the `MaskRCNNPredictor` output predictions for 91 classes. Specifically:

- `FastRCNNPredictor` ends with two Linear layers: one for the class scores ($[N, 91]$) and one for bounding box coordinates ($[N, 364]$, since 91×4).
- `MaskRCNNPredictor` outputs one segmentation mask per class, resulting in an output of shape $[N, 91, 28, 28]$.

Each of these 28×28 masks is a binary mask. This way, the model learns to shape the object within the region of interest by predicting values close to 1 for pixels inside the object and 0 for the background.

As mentioned at the start of this assignment, we will leverage a small dataset consisting of balloon images. This dataset is available in the link

<https://umassboston.instructure.com/courses/13964/files/folder/Assignment%203>

which contains two folders: one for training images and another for validation/testing.

Task 1: Create a PyTorch `Dataset` class tailored specifically for the balloon images provided. Additionally, implement a corresponding `DataLoader` that is ready to use for training and validation purposes. If you are unfamiliar with PyTorch's `Dataset` and `DataLoader` classes, please refer briefly to the official PyTorch documentation to gain a superficial understanding. You can find an initial code skeleton for the `Dataset` class to guide your implementation in the Files section on Canvas. Make some plots of the images and their corresponding masks to make sure you are loading them correctly. *Note: Typically, we'd distinguish clearly between validation and test sets, especially when performing hyperparameter optimization. However, for the scope of this assignment, this distinction won't be necessary.*

Task 2: Modify the Mask R-CNN network architecture to predict only two classes: the background and the balloon object. Print the updated model to verify the applied changes. *Hint:* Utilize the `FastRCNNPredictor` and `MaskRCNNPredictor` classes to replace `model.roi_heads.box_predictor` and `model.roi_heads.mask_predictor`. Note that the input size for the box predictor is 1024, and the mask predictor is 256.

Task 3: Fine-tune the modified Mask R-CNN model using the provided training subset. Starting codes for the training and validation loops are provided in the Files section on Canvas.

Task 4: Evaluate and compare the results before and after fine-tuning the modified architecture. Identify an example where the initial model failed but now correctly segments the object after fine-tuning. Visualize the differences clearly by plotting the masks on top of the images or using other effective visualization techniques.

Submission Instructions:

- **Format:** Submit your work as a **Jupyter Notebook (.ipynb)** file, ensuring all code cells are executable. Additionally, export the executed notebook as a **PDF or HTML report**, where all outputs and results are visible.

- **Structure of Submission:** Each task in your assignment should follow this structured approach:

- a. Introduction and objectives (briefly describe the purpose of the assignment)
- b. Implementation with justification (explain your solution and justify any design decision)
- c. Results and analysis ()
- d. Discussion of findings ()

At the end of the report, provide a conclusion summarizing key findings and reflecting on the effectiveness of the approaches used, highlighting their strengths and limitations.

- e. Conclusions

Evaluation Criteria:

Your assignment will be evaluated based on the following criteria:

- 1. Introduction and objectives (5%)**
 - Clearly state the purpose of the assignment.
 - Explain what you are doing and why it is important. Provide context or background information if necessary.
- 2. Implementation with justification (25%)**
 - Present a well-structured and efficient implementation. Ensure proper documentation and readability of the code.
 - Justify every critical decision made in the code. Explain the logic behind your approach, choice of algorithms, tools, or frameworks.
- 3. Analysis of the results (30%)**
 - Explain the results in relation to the objectives. Present them clearly, using plots, graphs, or explanations as needed.
 - Highlight any patterns, trends, or key insights from the findings.
- 4. Discussion of findings (40%)**
 - Critically evaluate the results and their implications.
 - Compare findings with expected outcomes (if applicable).
 - Discuss any limitations or challenges faced during the process. Provide suggestions for improvements or future work.

Good luck and have fun!